WEEK 1- DATA 56000

Spring 2026

Pol Monné Parera (L30170743)
polmonneparera@lewisu.edu

1. Technical Description

1.1 Backpropagation Neural Network

A backpropagation neural network (BPNN) is a supervised, fully connected multilayer perceptron (MLP) trained via the backpropagation of errors algorithm. It has an input layer, one or more hidden layers and, finally, an output layer. During the forward stage (inference) information flows and during the training errors flow backward.

The network used in this assignment has the following architecture:

- Input layer: 3 neurons (R, G, B pixel values, normalized to [0, 1])
- Hidden layer: 12 neurons with Sigmoid activation
- Output layer: 3 neurons (one per class) with SoftMax activation

1.2 Activation Functions

Sigmoid (hidden layer):
Smooth, differentiable, squashes activations to (0, 1). Provides the non-linearity needed for the network to learn complex boundaries.

SoftMax (output layer):
Converts raw scores to a proper probability distribution over the 3 classes, enabling cross-entropy loss

1.3 Pixel Classification Task

Each pixel in the image is treated as an independent sample with 3 features (R, G, B). The network learns to assign each pixel to one of three semantic color classes: Light/Background, Midtones, or Dark/Shadow.

The formulation used to assign pixels to each one of the three classes first starts by obtaining the luminescence via ITU-R BT.601 (a standard for converting RGB to luminance). It then assigns each class with the following thresholds which I have chosen arbitrarily by dividing the ranges in three equal ranges:

Class	Name	Luminance range
0	Light / Background	$Y \geq 170$
1	Midtones	$85 \leq Y < 170$
2	Dark / Shadow	$Y < 85$

2. Design of the Algorithms

2.1 Overall Pipeline

The pipeline implemented in the notebook can be observed in the steps below, with each step representing an independent stage of the code execution.

1. Load image (128×128 color crop), normalize pixels to [0,1]
2. Assign per-pixel class labels via luminance thresholds

3. Split into 80% train / 10% validation / 10% test
4. Initialize network weights
5. Train via backpropagation with early stopping
6. Evaluate on held-out test set, compute confusion matrix
7. Generate classification map for full image

2.2 Backpropagation Pseudocode

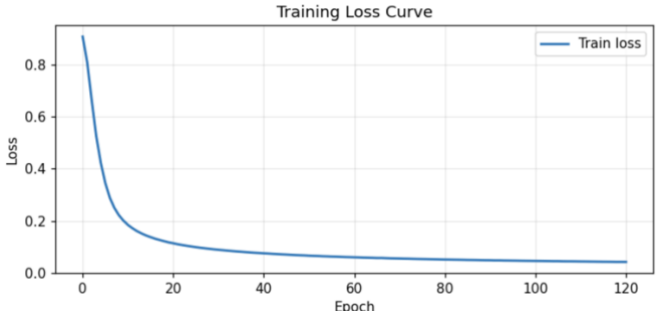
Below you will find a schema of the backpropagation algorithm implemented designed in a pseudocode style:

```
Initialize W1, b1, W2, b2 randomly
FOR epoch = 1 to MAX_EPOCHS:
    Shuffle training set
    FOR each batch B of size 512:
        // Forward stage
        z1 = X_B · W1 + b1
        a1 = sigmoid(z1)
        z2 = a1 · W2 + b2
        a2 = softmax(z2)

        // Backward stage
        δ2 = (a2 - Y_B) / m
        dW2 = a1^T · δ2
        δ1 = (δ2 · W2^T) ∗ sigmoid'(a1)
        dW1 = X_B^T · δ1

        // Parameter update
        W2 -= α · dW2; b2 -= α · db2
        W1 -= α · dW1; b1 -= α · db1
    IF val_loss not improving for PATIENCE epochs: BREAK
```

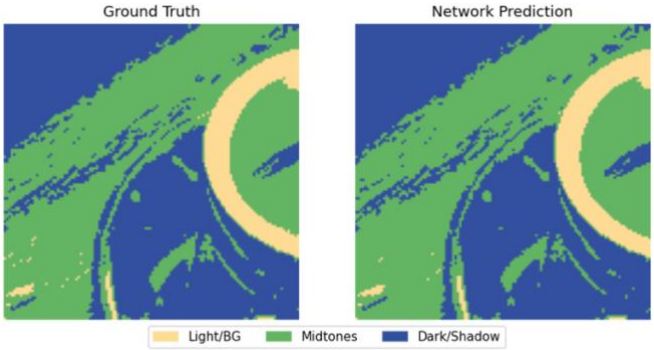
3. Results of the Algorithms

3.1 Original Image

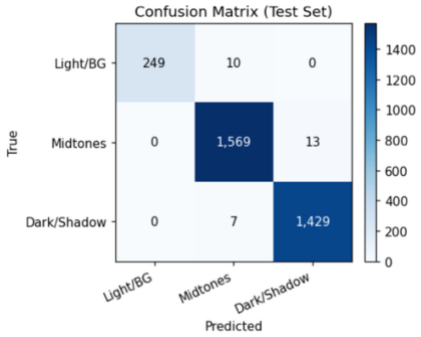
3.2 Training Loss Curve



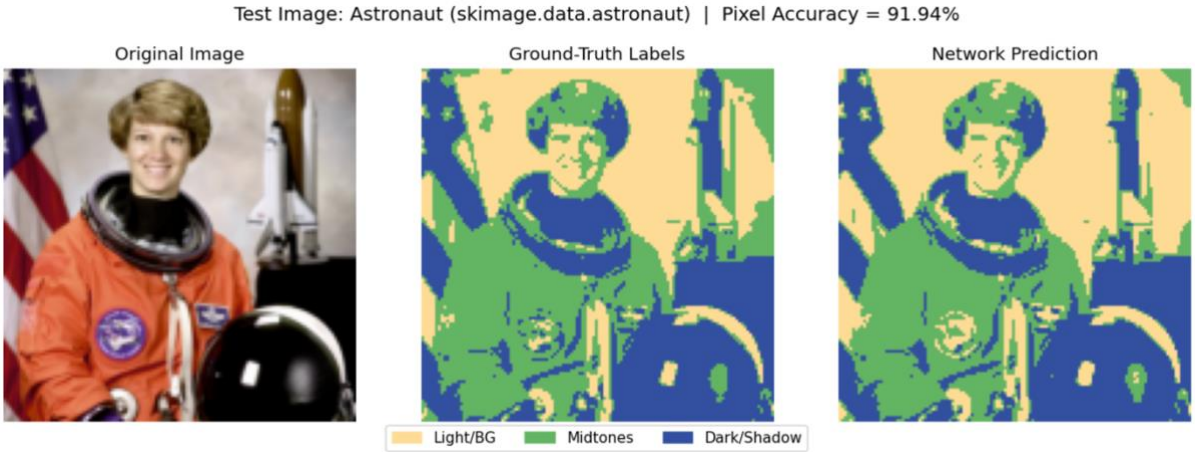
3.3 Classification Map

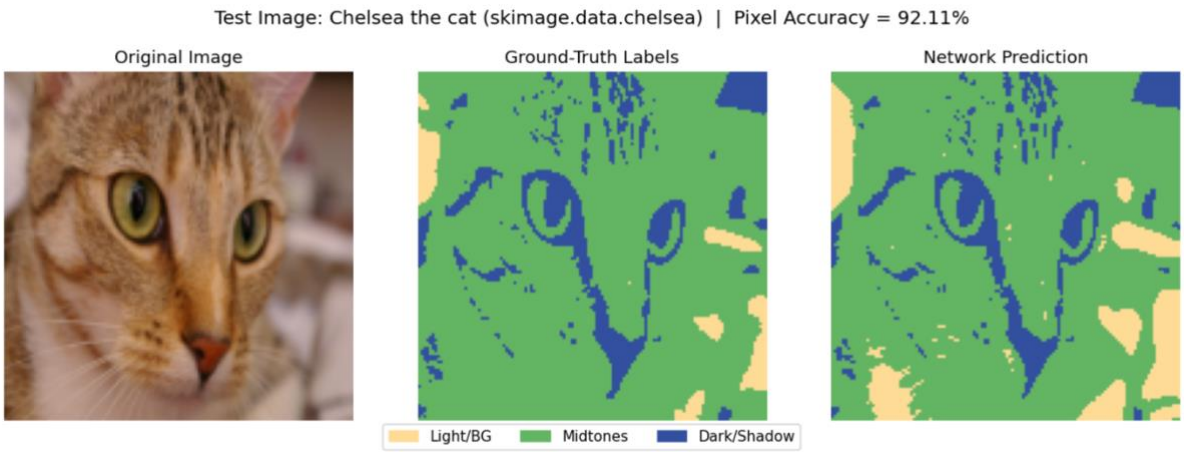


3.4 Confusion Matrix



3.5 Generalization





4. Analysis of the Results

4.1 Did I obtain what I expected?

The network achieved a high performing accuracy value (98.72%) which is way better that what I initially expected. That being said, it is true that given that the classification task is based on per-pixel RGB color values and the class boundaries are defined by luminance thresholds, the network has enough inductive bias to learn the decision boundaries well.

On the images loaded for generalization the results obtained were high (over 90% accuracy), but not as high as the original image. Given that the backpropagation neural network was trained with data from the original image it was expected that the model could perform slightly worse in unseen data, but it did actually perform quite well and close to the original values.

4.2 Surprises

The most notable surprise was how fast the network converged, the training loss curve shows how the model had already started converging by the hundredth epoch. I believe by my own research and logic that this fast learning can be attributed to the simplicity of the RGB color space.

4.3 Conclusions

The most surprising and relevant conclusion that I have been able to uncover is how even a simple and shallow network (only one hidden layer) with a small number of neurons can achieve high results (near perfect) on classification tasks.

It was also interesting to observe how the confusion matrix reveals the asymmetry in difficulty across classes: rare classes (Light/BG at 7.9%) are harder to classify correctly in recall, a common challenge in imbalanced datasets. Nevertheless, the network performs relatively well on all stages.

Seeing that the model did generalize correctly, but results were slightly worse on new images it would be a good following step to implement a training with a higher volume of data. With this approach the model would likely improve its overall accuracy and perform better with similar luminosity values.

5. Program Listings

The most relevant stages of the implementation are structured into the modular cells below:

```
X_tr, X_te, y_tr, y_te = train_test_split(
    X_all, y_all, test_size=0.20, random_state=42, stratify=y_all)
X_tr, X_val, y_tr, y_val = train_test_split(
    X_tr, y_tr,   test_size=0.10, random_state=42, stratify=y_tr)

print(f'Train : {len(X_tr):,}')
print(f'Val   : {len(X_val):,}')
print(f'Test  : {len(X_te):,}')
```

Figure 1 Data Splitting

```
IMAGE_SIZE  = 128
CLASS_NAMES  = ['Light/BG', 'Midtones', 'Dark/Shadow']
CLASS_COLORS = np.array([[255, 220, 150],
                         [100, 180, 100],
                         [ 50,  80, 160]])

nn = MLPClassifier(
    hidden_layer_sizes=(12,),
    activation='logistic',
    solver='sgd',
    learning_rate_init=0.15,
    batch_size=512,
    max_iter=600,
    early_stopping=True,
    validation_fraction=0.10,
    n_iter_no_change=30,
    tol=1e-4,
    random_state=42,
    verbose=True
)
print('Network configured.')
```

Figure 2 Hyperparameter Configuration

```
fig, ax = plt.subplots(figsize=(5, 4))
im = ax.imshow(cm, cmap='Blues')
ax.set_xticks(range(3)); ax.set_yticks(range(3))
ax.set_xticklabels(CLASS_NAMES, rotation=25, ha='right')
ax.set_yticklabels(CLASS_NAMES)
ax.set_xlabel('Predicted'); ax.set_ylabel('True')
ax.set_title('Confusion Matrix (Test Set)')
thresh = cm.max() / 2
for i in range(3):
    for j in range(3):
        ax.text(j, i, f'{cm[i,j]:,}', ha='center', va='center',
                color='white' if cm[i,j] > thresh else 'black')
fig.colorbar(im, ax=ax, fraction=0.046, pad=0.04)
plt.tight_layout()
plt.show()
```

Figure 3 Confusion Matrix

Resources

Image used: scikit-image built-in dataset — data.coffee(), a 400×600×3 color photograph of a coffee cup. Available at: https://scikit-image.org/docs/stable/api/skimage.data.html#skimage.data.coffee.

Image used: scikit-image built-in dataset — data.astronaut(), a 512×512×3 color photograph of astronaut Eileen Collins. Available at: https://scikit-image.org/docs/stable/api/skimage.data.html#skimage.data.astronaut

Image used: scikit-image built-in dataset — data.chelsea(), a 300×451×3 color photograph of a cat. Available at: https://scikit-image.org/docs/stable/api/skimage.data.html#skimage.data.chelsea